

HyperCom Overview

```
library(HyperCom)
```

```
library(dplyr)
#>
#> Attaching package: 'dplyr'
#> The following objects are masked from 'package:stats':
#>
#>     filter, lag
#> The following objects are masked from 'package:base':
#>
#>     intersect, setdiff, setequal, union
library(ggplot2)
```

Load in example data

HyperCom requires three inputs:

1. A dense gene x cell matrix with genes as row names and cells as column names
2. A dataframe of cell metadata containing sample and optionally group columns which need to be specified as variables
3. A dataframe of ligand-receptor (LR) interactions. HyperCom contains 2 built in LR databases formatted properly:
 - CellChat human `cellchat.hs.lrd`
 - CellChat mouse `cellchat.mm.lrd`

In this vignette we will use a 1000 cell subset of heart failure from Alexanian et al, 2021 (<https://doi.org/10.1038/s41586-021-03674-1>). This example subset is included in the package as `alexanian2021.counts` and `alexanian2021.metadata`.

```
counts <- alexanian2021.counts
dim(counts)
#> [1] 17083 1000
head(rownames(counts))
#> [1] "Xkr4" "Sox17" "Mrpl15" "Lypla1" "Tcea1" "Rgs20"
head(colnames(counts))
#> [1] "AACCGCGTCCAGAGGA-2" "AACTCAGCACGAGGTA-2" "AACTCTTGTCGAAAC-2"
#> [4] "AAGGAGCAGGATCGCA-2" "ACACCAAAGACCCACC-2" "ACACCGGAGCTACCGC-2"
```

```

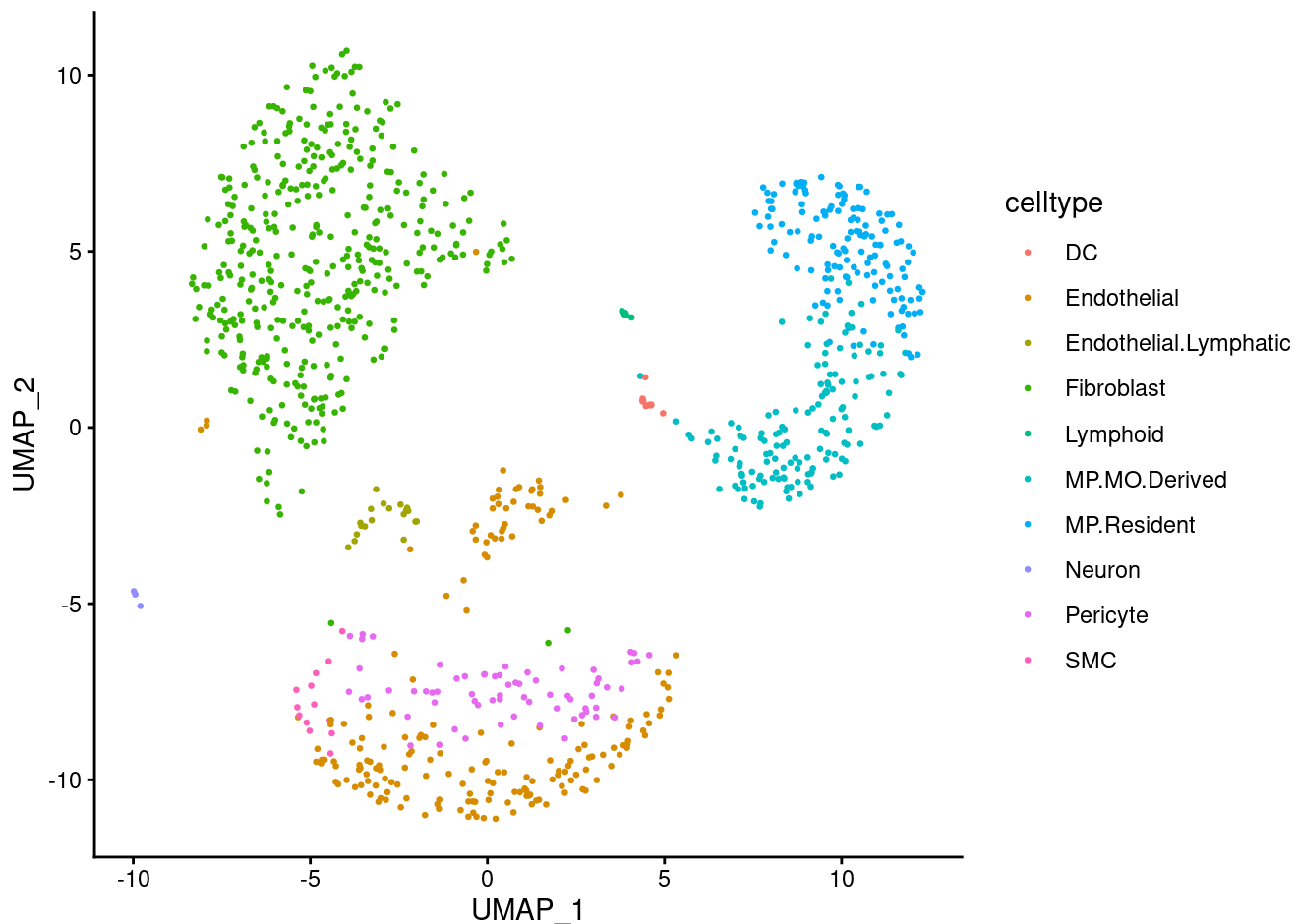
meta.data <- alexanian2021.metadata
sample.column <- "sample"
group.column <- "condition"
head(meta.data)
#>
#>      sample condition celltype    UMAP_1    UMAP_2
#> AACC GCGTCCAGAGGA-2      2    SHAM Endothelial  1.2676813 -10.036637
#> AACTCAGCACGAGGTA-2      2    SHAM Endothelial  1.7707077  -9.448688
#> AACTCTTGTCGAAAC-2      2    SHAM Endothelial  0.2925013  -9.781118
#> AAGGAGCAGGATCGCA-2      2    SHAM Endothelial  1.1058826 -10.454998
#> ACACCAAAGACCCACC-2      2    SHAM Endothelial -3.3748763 -10.034042
#> ACACCGGAGCTACCGC-2      2    SHAM Endothelial  3.9700553  -8.899605

```

```

ggplot(meta.data, aes(x=UMAP_1, y=UMAP_2, colour=celltype)) +
  geom_point(size=0.5) +
  theme_classic()

```



For the LR interactions we will use the CellChat murine database.

```

lrdb <- cellchat.mm.lrdb
dim(lrdb)
#> [1] 2377    6

```

```
head(lrdb)
#>      interaction ligand      r1      r2      r3      r4
#> 1  TGFB1_TGFB1_TGFB2  Tgfb1 Tgfbr1 Tgfbr2 <NA> <NA>
#> 2  TGFB2_TGFB1_TGFB2  Tgfb2 Tgfbr1 Tgfbr2 <NA> <NA>
#> 3  TGFB3_TGFB1_TGFB2  Tgfb3 Tgfbr1 Tgfbr2 <NA> <NA>
#> 4  TGFB1_ACVR1B_TGFB2  Tgfb1 Acvr1b Tgfbr2 <NA> <NA>
#> 5  TGFB1_ACVR1C_TGFB2  Tgfb1 Acvr1c Tgfbr2 <NA> <NA>
#> 6  TGFB2_ACVR1B_TGFB2  Tgfb2 Acvr1b Tgfbr2 <NA> <NA>
```

Running HyperCom

Preprocessing

In order to reduce memory usage, we first filter the counts matrix to only genes present in the LR interactions with expression greater than 0. The function also transposes the matrix.

```
lr.counts <- filter_counts(counts=counts, lrdb=lrdb, outfile=NULL)
dim(lr.counts)
#> [1] 1000  663
```

Next we filter the ligand receptor interactions to only interactions where all genes are present in the filtered expression matrix.

```
lrs <- generate_lrs_table(counts=lr.counts, lrdb=lrdb, outfile=NULL)
dim(lrs)
#> [1] 1042    5
```

Generating Hypergraph

To generate the hyperedges, which are edges that contain multiple nodes, we use the expression matrix LR interaction to create 3 types of hyperedges:

1. expression edges - edges containing cells that express a given ligand or receptor
2. complex edges - edges containing the genes that comprise a receptor complex
3. interaction edges - edges containing the genes that comprise an LR interaction

HyperCom represents these hyperedges as a dataframe with an edge and node column where the node belongs to the edge. The weight is set to 1 for each edge.

```
hyperedgelist <- generate_hyperedges(counts=lr.counts, lrs=lrs, outfile=NULL)
head(hyperedgelist)
#> # A tibble: 6 × 3
#>   edge          node weight
#>   <chr>         <chr>   <dbl>
#> 1 TGFB1_TGFB1_TGFB2 Tgfb1     1
#> 2 TGFB1_TGFB1_TGFB2 Tgfb1     1
#> 3 TGFB1_TGFB1_TGFB2 Tgfb2     1
#> 4 TGFB2_TGFB1_TGFB2 Tgfb2     1
#> 5 TGFB2_TGFB1_TGFB2 Tgfb1     1
#> 6 TGFB2_TGFB1_TGFB2 Tgfb2     1
```

We can represent this hypergraph as an adjacency matrix where the nodes are the cells, ligand genes, and receptor genes. Each entry is the number hyperedges shared by nodes.

```
adj <- generate_adjacency_matrix(hyperedgelist=hyperedgelist, outfile=NULL)
dim(adj)
#> [1] 1663 1663
```

Calculating Network Connectivity

In order to calculate the network connectivity of the hypergraph we perform a random walk on the adjacency matrix to produce an influence matrix.

```
infMat <- calculate_infMat(adj=adj, outfile=NULL)
dim(infMat)
#> [1] 1663 1663
```

Downstream Analysis

To account for genes now being present in the matrices, we then process the metadata to contain information on the genes as well as cell metadata. Additionally to reduce memory, we reduce the number of columns to just node, sample, and group. For this vignettes' visualization, we will also keep UMAP_1 and UMAP_2

```
metadata <- generate_metadata(infMat=infMat, metadata=meta.data, sample=sample.column, group=group.column, keep=c("UMAP_1", "UMAP_2"), outfile=NULL)
head(metadata)
#>
#>   cell sample group          UMAP_1          UMAP_2
#> 1 AAAGATGCACAGATTC-3      3   TAC  -4.5238003957282  4.36718559606172
#> 2 AAAGATGCAGCTCGCA-3      3   TAC  -3.88873865485295  7.9653258357773
#> 3 AAAGATGCAGGTCTCG-3      3   TAC  -3.70451190352544  3.86978268964387
#> 4 AAAGATGGTGTGTTGGT-2      2  SHAM  2.21441981911555 -2.05781912462614
#> 5 AAAGTAGAGGAGTTTA-2      2  SHAM -5.07207348227605  6.56972980840303
#> 6 AAAGTAGCACATTTCT-2      2  SHAM  1.42244932770625 -2.33817195551299
```

Scoring Network Activity

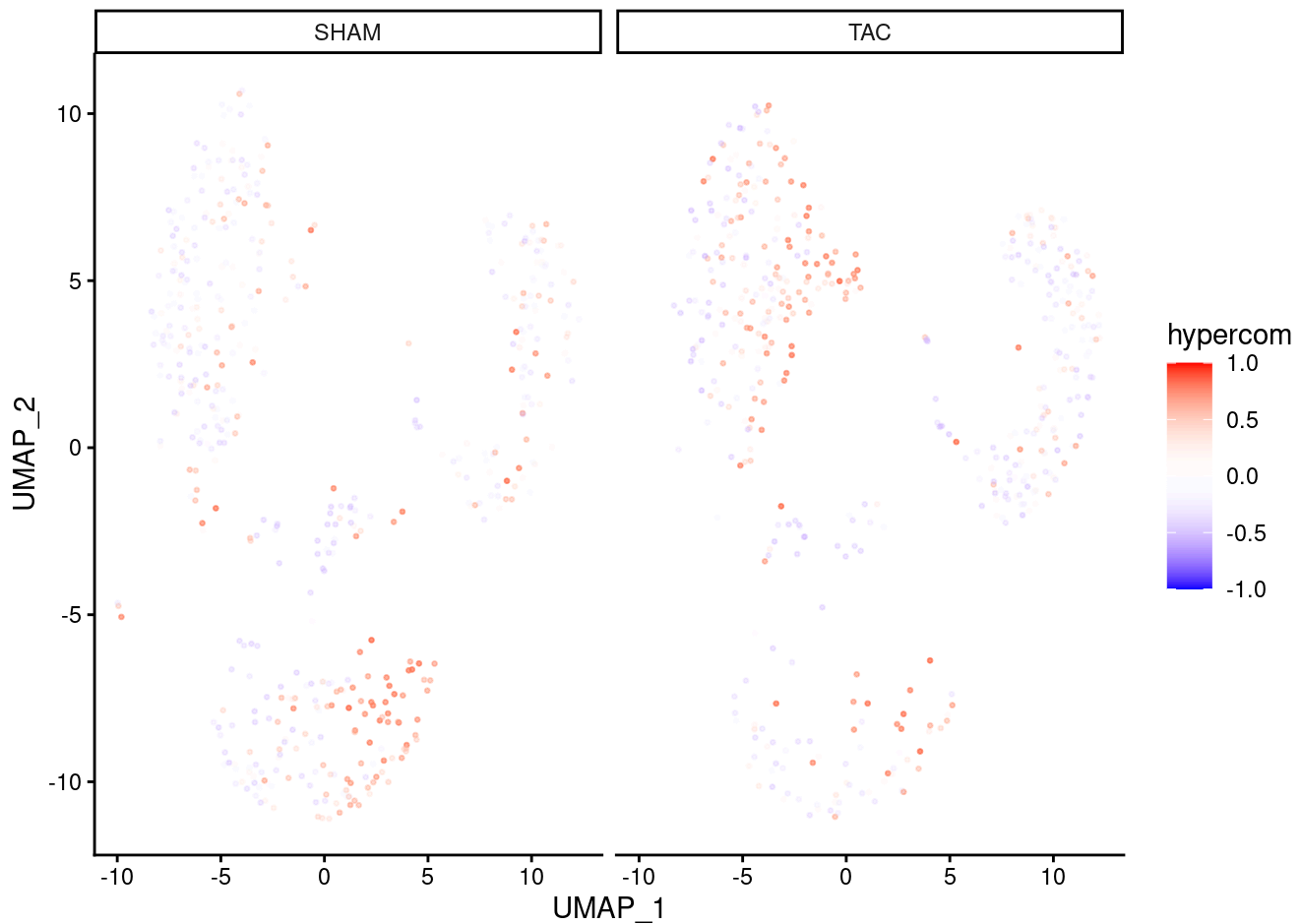
We can score network activity using the influence matrix to understand globally if cells are sending or receiving signals. HyperCom scoring generates a sender and receiver score based on a given matrix and uses both to create a composite HyperCom score. This HyperCom score ranges from -1 to 1 going from better senders to better receivers.

```
network.score <- score_hypercom(infMat=infMat, metadata=metadata)
head(network.score)
#>           cell      sender receiver      dif      avg hypercom
#> 1 GGGACCTGTGGGTATG-3 0.5554141 0.9671508 0.4117368 0.7612825 0.7368970
#> 2 GCTTCCAAGCCTCGTG-3 0.5386332 0.9659989 0.4273656 0.7523160 0.7327363
#> 3 CATTATCTCCGCTGTT-3 0.4904711 0.9646571 0.4741860 0.7275641 0.7239227
#> 4 ACACTGAGTTGTGGCC-3 0.5680951 0.9575791 0.3894840 0.7628371 0.7238935
#> 5 CCATGTCCACAGGCCT-3 0.5206136 0.9546570 0.4340433 0.7376353 0.7236062
#> 6 ACGAGCCAGAGGTTGC-2 0.4856558 0.9585472 0.4728914 0.7221015 0.7170417
```

Here we can see that globally, communication is higher in TAC.

```
network.score <- left_join(network.score, metadata, by=join_by(cell))

transparent <- rgb(255, 255, 255, max=255, alpha = 0)
ggplot(network.score, aes(x=as.numeric(UMAP_1), y=as.numeric(UMAP_2), colour=hypercom)) +
  geom_point(size=0.5) +
  scale_color_gradient2(low="blue", mid=transparent, high="red", limits=c(-1,1)) +
  facet_wrap(vars(group)) +
  xlab("UMAP_1") +
  ylab("UMAP_2") +
  theme_classic()
```



Identifying Important LR Pairs

We can also identify important LR pairs by using HyperCom scoring on LR pairs. This prioritization scores LR pairs to find highly communicative LR pairs that are optionally group specific. The `prioritize_lr` function can be run sequentially or parallelly. The parallel implementation uses the `parallelly` package.

```
priority <- prioritize_lr(lrs=lrs, adj=adj, infMat=infMat, metadata=metadata, parallel=TRUE, cl=
NULL, outfile=NULL)
head(priority)
```

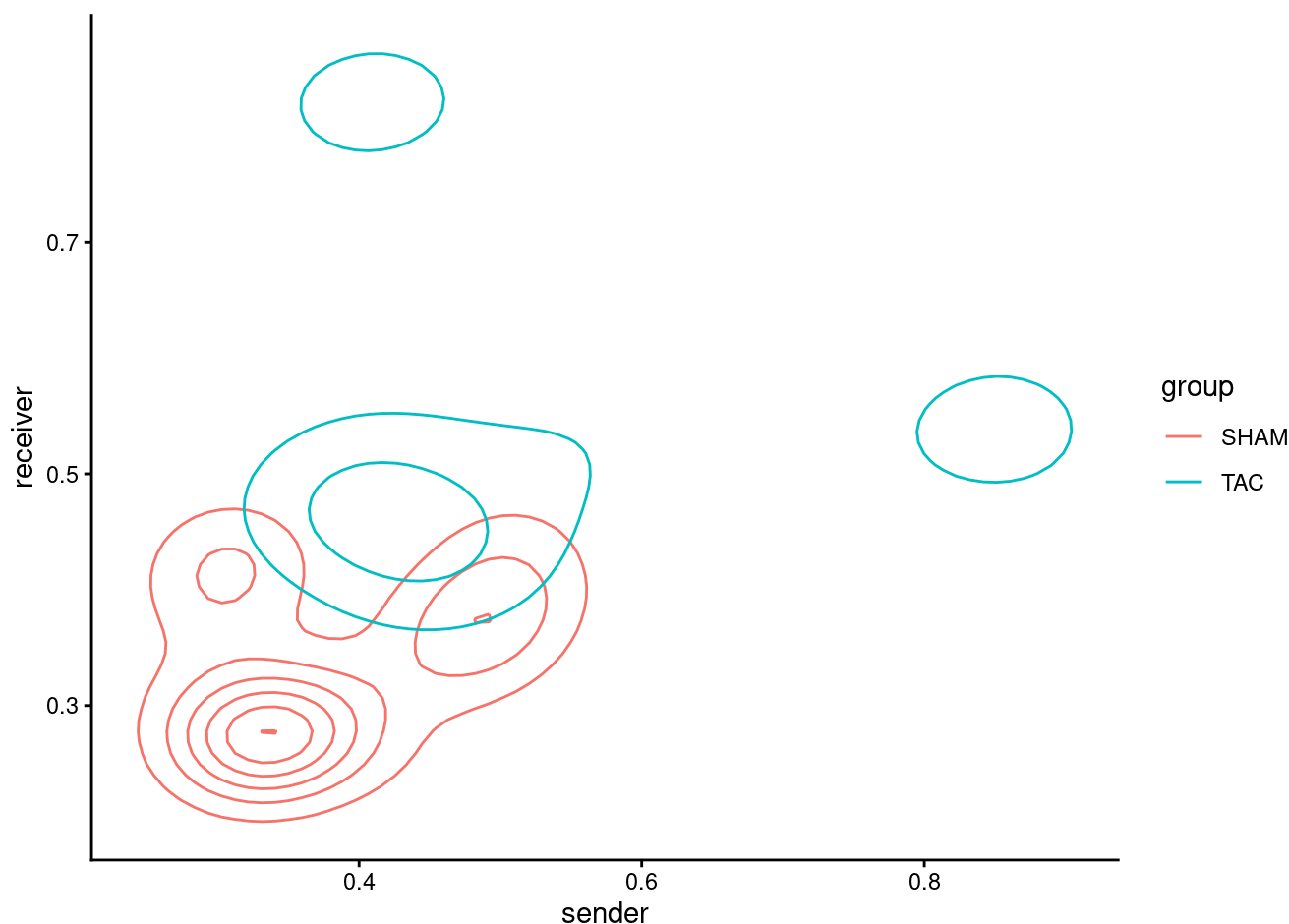
#>	interaction	ligand	r1	r2	r3	priority
#> 1	THBS1_SDC4	Thbs1	Sdc4	<NA>	<NA>	0.3307897
#> 2	PGF_VEGFR1	Pgf	Flt1	<NA>	<NA>	0.3173039
#> 3	COMP_SDC4	Comp	Sdc4	<NA>	<NA>	0.2917878
#> 4	POSTN_ITGAV_ITGB5	Postn	Itgav	Itgb5	<NA>	0.2915502
#> 5	FN1_SDC4	Fn1	Sdc4	<NA>	<NA>	0.2711980
#> 6	FN1_CD44	Fn1	Cd44	<NA>	<NA>	0.2691572

We see that `Thbs1 Sdc4` is the highest scoring LR pair. We can calculate HyperCom scores for this pair. As a note, the `score_hypercom` function can also take a vector of receptor genes if the receptor is a complex.

```
Thbs1_Sdc4 <- score_hypercom(infMat=infMat, metadata=metadata, ligand="Thbs1", receptors="Sdc4")
head(Thbs1_Sdc4)
#>      cell      sender receiver      dif      avg hypercom
#> 1 ATTATCCAGCTACCGC-3 0.4555440 0.9485435 0.4929995 0.7020437 0.6893943
#> 2 ACTGTCCCAATAACGA-2 0.4912161 0.9117634 0.4205473 0.7014897 0.6312705
#> 3 CGTTAGAGTTACGCGC-3 0.4744556 0.8981270 0.4236714 0.6862913 0.6244633
#> 4 CTGAAGTAGTCGTTTG-3 0.4051788 0.8627810 0.4576022 0.6339799 0.6079607
#> 5 CCCAGTTTCATGCTCC-3 0.3959453 0.8545903 0.4586450 0.6252678 0.6008580
#> 6 AGGTCCGGTGTTGGGA-2 0.5717083 0.9045992 0.3328908 0.7381538 0.5999808
```

If we plot the sender and receiver score we see both higher densities of sender and receiver cells in TAC.

```
Thbs1_Sdc4 <- left_join(Thbs1_Sdc4, metadata, by=join_by(cell))
ggplot(Thbs1_Sdc4, aes(x=sender, y=receiver, colour=group)) +
  geom_density_2d() +
  theme_classic()
```



Side Note: Communication Clusters

Although clustering is not necessary for any step in HyperCom, it may be useful to conceptualize how groups of cells are communicating. We can cluster the influence matrix to generate communication clusters. Here we use Seurat's implementation of Louvain clustering.

```

library(Seurat)
#> Loading required package: SeuratObject
#> Loading required package: sp
#>
#> Attaching package: 'SeuratObject'
#> The following objects are masked from 'package:base':
#>
#> intersect, t
generate_com_clusters <- function(infMat, metadata){
  com.seurat <- CreateSeuratObject(t(infMat), meta.data = metadata, project="communication")
  com.seurat <- NormalizeData(com.seurat, verbose=FALSE)
  com.seurat <- FindVariableFeatures(com.seurat, selection.method = "vst", nfeatures = 2000, verbose=FALSE)
  all.genes <- rownames(com.seurat)
  com.seurat <- ScaleData(com.seurat, features = all.genes, verbose=FALSE)
  com.seurat <- RunPCA(com.seurat, features = VariableFeatures(object = com.seurat), verbose=FALSE)
  com.seurat <- FindNeighbors(com.seurat, dims = 1:10, verbose=FALSE)
  com.seurat <- FindClusters(com.seurat, resolution = 0.5, cluster.name = "com_clusters", verbose=FALSE)
  com.seurat <- RunUMAP(com.seurat, dims = 1:10, verbose=FALSE)

  hyper.com_clusters <- dplyr::select(com.seurat@meta.data, cell, com_clusters)
  metadata <- left_join(metadata, hyper.com_clusters, by=join_by(cell))
  metadata$com_clusters <- factor(metadata$com_clusters)

  return(metadata)
}

```

We can see that communication clusters do not necessarily line up with the annotated cell types.

```

com_clusters <- generate_com_clusters(infMat=infMat, metadata=metadata)
#> Warning: Data is of class matrix. Coercing to dgCMatrix.
#> Warning: The default method for RunUMAP has changed from calling Python UMAP via reticulate to the R-native UWOT using the cosine metric
#> To use Python UMAP via reticulate, set umap.method to 'umap-learn' and metric to 'correlation'
#> This message will be shown once per session

annotations <- select(meta.data, celltype)
annotations$cell <- rownames(annotations)
com_clusters <- left_join(com_clusters, annotations, by=join_by(cell))
com_clusters <- filter(com_clusters, group != "gene")

ggplot(com_clusters, aes(x=com_clusters, fill=celltype)) +
  geom_bar() +
  theme_classic()

```

